Аутентификация пользователей в веб-приложениях

Как сервер узнаёт пользователя и поддерживает аутентифицированное состояние

Результаты занятия

- [x] Объяснять назначение аутентификации
- [x] Различать Authentication и Authorization
- [x] Понимать роль identifier и credential
- [x] Объяснять процессы регистрации и входа
- [x] Понимать проблему stateless HTTP
- [x] Различать session-based и token-based authentication на концептуальном уровне
- [x] Проектировать /api/auth/register и /api/auth/login
- [x] Не возвращать passwordHash клиенту
- [x] Использовать одинаковые ошибки для неверных учётных данных
- [x] Связывать frontend-форму входа с Express REST API

Учебные вопросы

1	**Что такое аутентификация**	Identity, credentials, Authentication и Authorization
2	**Регистрация и вход**	User model, register/login endpoints и Service
3	**Аутентифицированное состояние**	Почему после login сервер должен узнавать пользователя снова
4	**Frontend ↔ Backend**	Форма входа, fetch(), ответы API и состояние интерфейса
5	**Базовые правила безопасности**	HTTPS, одинаковые ошибки, секреты и безопасное хранение паролей

Учебный вопрос 1. Что такое аутентификация

Сначала устанавливаем личность пользователя, затем решаем, что ему разрешено

Authentication

- Аутентификация — процесс проверки того, что пользователь действительно является владельцем заявленной учётной записи.
- Пользователь сообщает идентификатор, например email.
- Пользователь предъявляет credential, например пароль.
- Сервер проверяет credential.
- При успехе приложение устанавливает аутентифицированное состояние.
- На следующих запросах сервер должен снова распознавать пользователя.

Authentication и Authorization

Authentication — AuthN

- Кто пользователь?
- Проверяет личность
- Login
- Email + пароль
- Создаёт аутентифицированный контекст

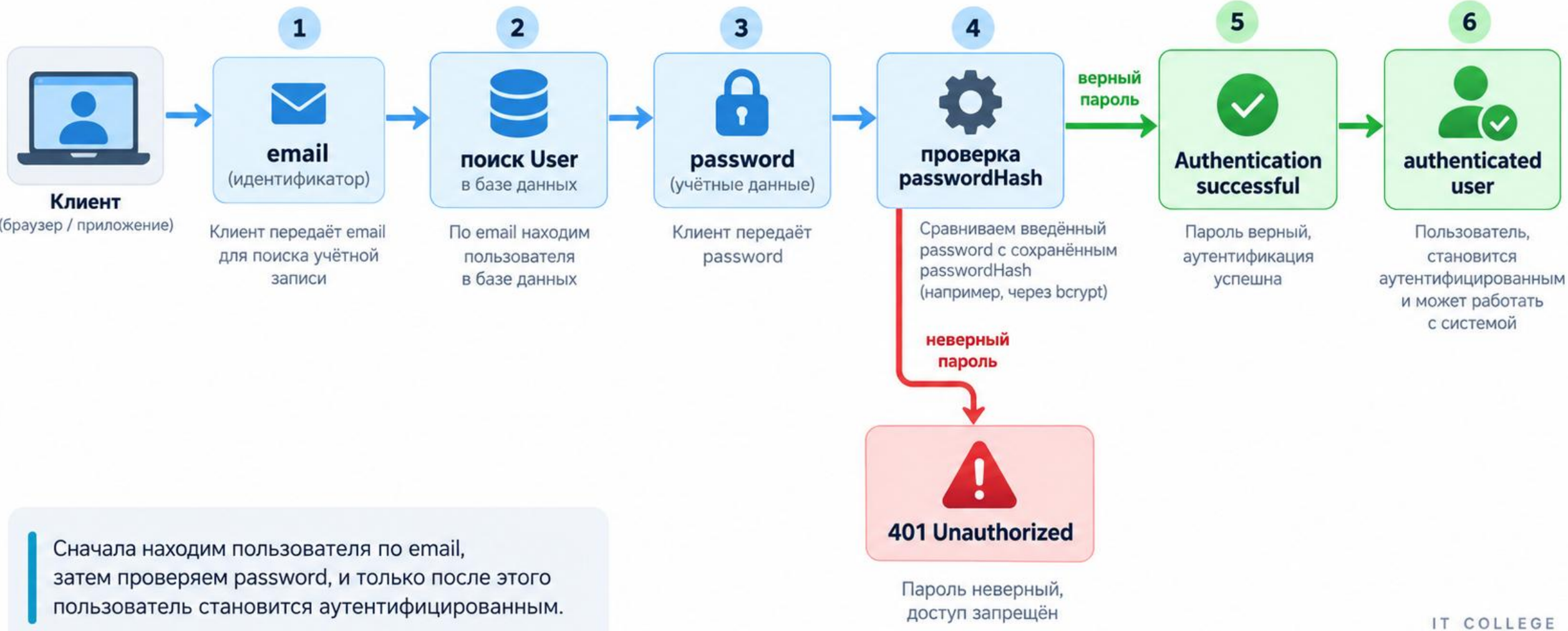
Authorization — AuthZ

- Что пользователю разрешено?
- Проверяет права
- Роли и разрешения
- Например admin/user
- Изучим отдельно

Основные понятия

Термин	Смысл	Пример Internet Shop
Identity	Цифровая идентичность пользователя	Учётная запись покупателя
Identifier	Значение, по которому ищем учётную запись	email
Credential	Секрет или фактор подтверждения	password
Authentication	Проверка credential	POST /api/auth/login
Authenticated state	Способ узнавать пользователя после login	Session ID или token

Путь аутентификации



Сначала находим пользователя по email, затем проверяем password, и только после этого пользователь становится аутентифицированным.

HTTP не помнит предыдущий запрос

- HTTP по своей природе stateless.
- POST /api/auth/login и следующий GET /api/orders — разные запросы.
- Сам факт успешного login не заставляет сервер автоматически помнить пользователя.
- Поэтому приложение использует session ID или token.
- Механизм состояния связывает последующие HTTP-запросы с аутентифицированным пользователем.

Жизненный цикл аутентификации

1	**Идентификация**	Клиент передаёт email или другой identifier.
2	**Проверка credential**	Сервер проверяет пароль или другой фактор.
3	**Установление состояния**	Создаётся session или выдаётся token.
4	**Следующие запросы**	Сервер восстанавливает identity пользователя.
5	**Logout или expiration**	Аутентифицированное состояние прекращается.

Факторы аутентификации

Knowledge

То, что пользователь знает: пароль, PIN.

Possession

То, чем пользователь владеет: телефон, аппаратный ключ.

Inherence

То, чем пользователь является: биометрия.

Учебный вопрос 2. Регистрация и вход

Два разных процесса: создать учётную запись и подтвердить существующую

User model для аутентификации

```prisma
model User {
  id           Int      @id @default(autoincrement())
  name         String
  email        String   @unique
  passwordHash String   @map("password_hash")
  role         String   @default("user")
  orders       Order[]

  @@map("users")
}
```

Что важно

- email используется как identifier.
- passwordHash хранит не исходный пароль, а результат безопасного password hashing.
- role понадобится для авторизации.
- @unique запрещает две учётные записи с одинаковым email.

Пароль и passwordHash — разные данные

- Пароль приходит от пользователя только при регистрации или входе.
- Исходный пароль нельзя сохранять в PostgreSQL.
- В базе хранится passwordHash.
- При login приложение сравнивает введённый пароль с hash.
- В этой лекции hash/verify используем как готовые операции; механизм подробно разберём на следующем занятии.

Регистрация пользователя

1	**Получить данные**	name, email, password приходят из frontend.
2	**Проверить формат**	Validation проверяет обязательные поля и типы.
3	**Проверить email**	Service убеждается, что email ещё не занят.
4	**Создать passwordHash**	Исходный password преобразуется безопасной функцией.
5	**Создать User**	В PostgreSQL сохраняются name, email и passwordHash.
6	**Вернуть безопасный ответ**	passwordHash клиенту не отправляется.

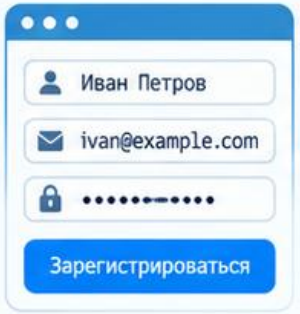
Регистрация: password приходит → passwordHash сохраняется

Пользователь отправляет пароль, мы хешируем его, и в базу данных сохраняется только хеш.

Безопасные веб-приложения начинаются с правильной обработки паролей

1 Frontend

Регистрационная форма



Пример данных:

```
{
  "name": "Иван Петров",
  "email": "ivan@example.com",
  "password": "Qwerty123!"
}
```

2 POST /api/auth/register

HTTP-запрос с данными регистрации



3 Validation

Проверка и валидация данных



- Корректный email
- Длина пароля
- Требования к сложности
- Уникальность email

4 Auth Controller

Обрабатывает запрос



Вызывает сервис регистрации

5 Auth Service

Бизнес-логика регистрации



Валидные данные → хеширование → сохранение в БД

6 hashPassword (password)

Преобразование пароля в хеш



bcrypt / argon2 (соль + хеш)

```
hashPassword("Qwerty123!")
→ $2b$10$...a8f3e9c1d4
```

7 passwordHash

Результат хеширования



Только хеш передается дальше

8 Prisma

Сохраняет данные в базу



Prisma

```
prisma.user.create({
  name,
  email,
  password_hash
})
```

9 users (PostgreSQL)

Таблица пользователей



Таблица users

id	UUID	(PK)
name	VARCHAR	
email	VARCHAR	(UNIQUE)
password_hash	VARCHAR	
role	VARCHAR	
created_at	TIMESTAMP	
...		

Прямое сохранение исходного password в базу данных НЕДОПУСТИМО!

НЕТ столбца "password" в таблице users!

Исходный password в PostgreSQL не сохраняется

Пароль преобразуется в хеш (например, с помощью bcrypt или argon2), и в базу данных записывается только passwordHash. Даже в случае утечки базы данных злоумышленники не получат исходные пароли пользователей.

Хеширование паролей — обязательная практика в современных веб-приложениях.

Вход пользователя

1	**Получить email и password**	Frontend отправляет credentials на login endpoint.
2	**Найти User**	Service ищет учётную запись по email.
3	**Проверить password**	Введённый пароль проверяется относительно passwordHash.
4	**Установить состояние**	После успеха создаётся session или token.
5	**Вернуть User**	Клиент получает безопасные данные пользователя.

Одинаковая ошибка при неверном login

- Не сообщайте отдельно, что email существует, но пароль неверен.
- Не сообщайте отдельно, что такого email нет.
- Оба случая должны выглядеть для внешнего клиента одинаково.
- Например: Invalid email or password.
- Это снижает возможность перебора существующих учётных записей.

Auth routes

JAVASCRIPT

```
router.post("/register", register);
router.post("/login", login);
router.post("/logout", logout);
router.get("/me", getCurrentUser);
```

Назначение Auth endpoints

- POST /api/auth/register — создать учётную запись.
- POST /api/auth/login — проверить credentials.
- GET /api/auth/me — получить текущего аутентифицированного пользователя.
- POST /api/auth/logout — завершить аутентифицированное состояние.
- Конкретная реализация /me и logout зависит от session/token механизма.

Register Controller

```javascript
export async function register(req, res, next) {
  try {
    const user = await registerUser(req.body);
    res.status(201).json(user);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- Controller получает HTTP body.
- Service выполняет логику регистрации.
- Controller не хеширует пароль.
- Успешная регистрация возвращает 201.

Register Service

```javascript
export async function registerUser({ name, email, password }) {
  const existing = await prisma.user.findUnique({ where: { email } });

  if (existing) {
    throw new Error("Email already registered");
  }

  const passwordHash = await hashPassword(password);

  return prisma.user.create({
    data: { name, email, passwordHash },
    select: { id: true, name: true, email: true, role: true }
  });
}
```

Что важно

- Service проверяет уникальность email.
- hashPassword() скрывает детали hashing.
- В Prisma сохраняется только passwordHash.
- select гарантирует, что passwordHash не попадёт в ответ.

Login Service

JAVASCRIPT

```
export async function authenticateUser(email, password) {
  const user = await prisma.user.findUnique({ where: { email } });

  if (!user) return null;

  const valid = await verifyPassword(user.passwordHash, password);
  if (!valid) return null;

  return {
    id: user.id,
    name: user.name,
    email: user.email,
    role: user.role
  };
}
```

Что важно

- По email находится User.
- verifyPassword() сравнивает password с passwordHash.
- При любой проблеме credentials Service возвращает null.
- passwordHash не возвращается Controller.

Учебный вопрос 3. Аутентифицированное состояние

После login нужно связать следующие запросы с User

Login — это не конец процесса

- Успешная проверка password подтверждает identity в конкретный момент.
- Следующий HTTP-запрос снова приходит отдельно.
- Серверу нужен надёжный признак аутентифицированного пользователя.
- Обычно используется session ID в cookie или bearer token.
- Этот признак становится ключом к восстановлению identity.

Session-based и token-based authentication

Session

- Состояние хранится на сервере
- Клиент хранит session ID
- Часто session ID передаётся cookie
- Logout инвалидирует session
- Удобно для классических web-приложений

Token

- Клиент предъявляет token
- Token может содержать утверждения о пользователе
- Часто передаётся как Bearer token
- Нужно управлять expiration и отзывом
- JWT подробно изучим отдельно

Session ID

- Session ID — случайный идентификатор серверной сессии.
- Сам session ID не должен содержать пароль и персональные данные.
- Сервер по session ID находит session и userId.
- Для browser-приложения session ID обычно помещают в защищённую cookie.
- После logout session должна стать недействительной.

Bearer token

- Token предъявляется вместе с запросом.
- Сервер проверяет token перед доступом к защищённому endpoint.
- В token нельзя помещать пароль.
- Token должен иметь срок действия.
- JWT и его проверку подробно разберём в лекции 3.

После login: Session ID или Bearer token

Успешная аутентификация
(Authentication successful)

Пользователь ввел логин и пароль. Сервер проверил учетные данные и аутентификация прошла успешно.

Сессионная аутентификация (Session)

На сервере хранится состояние сессии, а в браузере — только session ID в cookie.

Session



Пользователь успешно вошел. Сервер создает сессию для пользователя.

session ID

`abc123...`

Генерируется уникальный идентификатор сессии (session ID).

cookie



Session ID сохраняется в cookie браузера.

серверное состояние



На сервере хранится состояние сессии (данные пользователя, права, время жизни и т.д.).

Токенная аутентификация (Token, Bearer)

В браузере хранится сам токен. Сервер проверяет его при каждом запросе.

Token



Пользователь успешно вошел. Сервер создает токен (например, JWT).

Bearer token

`eyJhbGciOiJIUzI1NiIs`
`InR5cCI6IkpXVCJ9...`

Токен выдается клиенту и сохраняется (например, в памяти или localStorage).

проверка сервером



При каждом запросе сервер проверяет токен (валидность, подпись, срок действия и т.д.).

Оба механизма связывают следующий HTTP request с User

Следующий HTTP request (сессия)



Браузер (клиент)

Отправляет cookie

`Cookie: session_id=abc123...`



Сервер (хранилище сессий)

Сервер находит сессию по session ID и определяет пользователя.

Следующий HTTP request (токен)



Браузер (клиент)

Отправляет Bearer token в заголовке Authorization

`Authorization: Bearer eyJhbGciOi...`



Сервер (валидация токена)

Сервер проверяет токен и определяет пользователя.

Защищённый endpoint

- Публичные endpoints доступны без аутентификации.
- Защищённые endpoints требуют подтверждённого User.
- Например, GET /api/profile или GET /api/orders/my.
- Проверка выполняется middleware до Controller.
- После успешной проверки middleware добавляет данные пользователя в req.

Концепция auth middleware

```javascript
export async function requireAuth(req, res, next) {
  const identity = await resolveIdentity(req);

  if (!identity) {
    return res.status(401).json({ error: "Authentication required" });
  }

  req.user = identity;
  next();
}
```

Что важно

- resolveIdentity() скрывает конкретный session/token механизм.
- Без identity запрос останавливается с 401.
- При успехе req.user доступен следующему middleware и Controller.
- Конкретную реализацию token verification изучим позже.

Route c requireAuth

```javascript
router.get("/me", requireAuth, getCurrentUser);
```

Logout

- Logout прекращает текущее аутентифицированное состояние.
- Для session-based схемы сервер удаляет или инвалидирует session.
- Для token-based схемы стратегия зависит от типа token и архитектуры.
- Frontend после logout очищает своё локальное состояние пользователя.
- Пароль при logout не передаётся.

Учебный вопрос 4. Frontend ↔ Backend

Форма входа — только начало: решение принимает сервер

Frontend отправляет login

JAVASCRIPT

```
const response = await fetch("http://localhost:3000/api/auth/login", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({ email, password })
});
```

Что важно

- Frontend собирает email и password из формы.
- Credentials отправляются только серверу.
- Frontend не должен самостоятельно решать, правильный ли пароль.
- Ответ API определяет дальнейшее состояние интерфейса.

Успешный login

Запрос: POST /api/auth/login

Код • JSON

```
{
  "email": "alex@example.com",
  "password": "ExamplePassword123!"
}
```

Результат • JSON

```
{
  "user": {
    "id": 5,
    "name": "Alex",
    "email": "alex@example.com",
    "role": "user"
  }
}
```

Пояснение

- В ответе нет password и passwordHash.

Неверные credentials

Запрос: POST /api/auth/login

Код • JSON

```
{
  "email": "alex@example.com",
  "password": "wrong-password"
}
```

Результат • JSON

```
{
  "error": "Invalid email or password"
}
```

Пояснение

- Такая же внешняя ошибка должна использоваться и для неизвестного email.

Frontend после login

1	**Получить response**	Frontend проверяет HTTP status.
2	**Получить user**	При успехе сохранить безопасные данные пользователя в состоянии интерфейса.
3	**Изменить UI**	Показать имя пользователя, профиль и logout.
4	**Работать с protected API**	Следующие запросы должны передавать session/token механизм.
5	**Обработать 401**	Показать форму входа или завершить локальную сессию интерфейса.

Frontend login → Auth API → User state

Express.js | Prisma | PostgreSQL

От формы входа на клиенте до получения данных пользователя и отображения профиля

1 HTML login form (Frontend)

Пользователь вводит email и пароль



2 fetch() запрос

Отправка данных на сервер

```
// login.js
const data = {
  email: emailInput.value,
  password: passwordInput.value
};

const res = await fetch(
  '/api/auth/login',
  {
    method: 'POST',
    headers: {
      'Content-Type':
       'application/json'
    },
    body: JSON.stringify(data)
  }
);

const result = await res.json();
```

JS Отправляем email и password в JSON формате

3 Auth API (Express.js)

Маршрут и контроллер

```
// routes/auth.js
router:post('/api/auth/login',
 authController.login);
```

Auth Controller

Обработка запроса

- Валидация данных
- Вызов authService.login()
- Формирование ответа

4 Auth Service

Проверка учётных данных

```
// authService.js
const user = await prisma.user
  .findUnique({ where: { email } });

if (!user) throw new Error('Invalid');

const isValid = await
  passwordService.verify(
    password, user.passwordHash
  );

if (!isValid) throw new Error('Invalid');

// Успешная аутентификация
return {
  id: user.id,
  email: user.email,
  name: user.name,
  role: user.role
};
```

Проверяем пользователя в БД и сравниваем пароль (passwordHash)

5 Prisma / Database

Поиск пользователя

Prisma Client

```
const user = await
  prisma.user.findUnique({
    where: { email }
  });
```

PostgreSQL

Таблица users

id	UUID
email	VARCHAR (UNIQUE)
passwordHash	VARCHAR
name	VARCHAR
role	VARCHAR
createdAt	TIMESTAMP

В БД хранится только хеш пароля, а не сам пароль

6 Response и User state

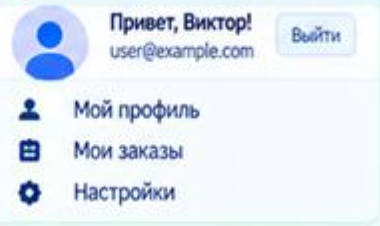
Возврат данных и отображение профиля на фронтенде

Успешный ответ (200 OK)

Безопасный JSON (без пароля)

```
{
  "user": {
    "id": "1",
    "email": "user@example.com",
    "name": "Виктор",
    "role": "user"
  }
}
```

Пользователь авторизован



Ошибка (401 Unaathorized)

Неверный email или пароль

```
{
  "error": "Invalid credentials"
}
```

Ключевые моменты

- Пароль передаётся по HTTPS
- На сервере выполняется валидация и проверка пароля
- В ответе не возвращается пароль (только безопасные данные)
- После входа можно сохранять сессию (cookie) или выдавать JWT

Безопасность

- Используйте HTTPS
- Одинаковое сообщение об ошибке (не раскрывать, существует ли пользователь)
- Ограничение попыток входа (rate limiting)
- Логирование подозрительной активности

Результат

- Пользователь успешно аутентифицирован
- Frontend получает данные пользователя
- Отображается профиль
- В следующих запросах пользователь определяется по сессии (cookie) или Bearer token (JWT)

Учебный вопрос 5. Базовые правила безопасности

Аутентификация работает с наиболее чувствительными данными приложения

Минимальные правила

- [x] Использовать HTTPS в production
- [x] Никогда не хранить пароль в открытом виде
- [x] Не отправлять passwordHash клиенту
- [x] Не писать passwords и tokens в логи
- [x] Использовать одинаковые ошибки login
- [x] Ограничивать частоту попыток входа
- [x] Использовать безопасные cookies для session-based схем
- [x] Завершать или обновлять аутентификацию после чувствительных событий

HTTPS нужен не только на странице login

- Credentials должны передаваться только по защищённому соединению.
- Session ID и tokens также являются чувствительными данными.
- После login защищённым должен оставаться весь authenticated traffic.
- Перехваченный session ID или token может позволить выдать себя за пользователя.

Не раскрывайте существование учётной записи

- Сообщение Email not found раскрывает информацию злоумышленнику.
- Сообщение Wrong password подтверждает существование email.
- Для login лучше использовать единый ответ Invalid email or password.
- Внутренние детали ошибки можно журналировать безопасно на сервере.
- Пароль и полный token в лог записывать нельзя.

Что ещё усиливает Authentication

Rate limiting

Ограничивает массовый перебор login/password.

MFA

Добавляет второй независимый фактор аутентификации.

Reauthentication

Повторно подтверждает identity перед чувствительной операцией.

Secure session

Защищает session ID или token после успешного login.

Мини-практика: Auth module Internet Shop

Создайте register/login API и убедитесь, что passwordHash никогда не возвращается клиенту.

Задание

- Добавить passwordHash в User
- Создать routes/authRoutes.js
- Создать controllers/authController.js
- Создать services/authService.js
- Реализовать POST /api/auth/register
- Реализовать POST /api/auth/login
- Для hashing использовать готовые hashPassword() и verifyPassword()
- Проверить одинаковую login-ошибку для неизвестного email и неверного password

Критерии проверки

- User создаётся в PostgreSQL
- В базе нет открытого password
- Login принимает email и password
- Правильные credentials возвращают безопасный User
- Неверные credentials возвращают 401
- passwordHash отсутствует во всех HTTP-ответах

Архитектура Authentication в Express Internet Shop

Express.js | Prisma | PostgreSQL

От клиента до базы данных: регистрация, вход и защита маршрутов

Frontend
Веб-клиент (Browser)



После входа:
- Сохраняет cookie (session) или Bearer token (JWT)
- Отправляет токен в заголовке Authorization

HTTP Request →

← JSON Response

Auth Routes
Маршруты аутентификации (Express Router)

express

```
// routes/auth.js
const router =
  require('express').Router();

router.post('/register',
  authController.register);

router.post('/login',
  authController.login);

// Защищённые маршруты
router.get('/me',
  requireAuth,
  authController.me);

// Экспорт маршрутов
module.exports = router;
```

Auth Controller
Обработка HTTP запросов

register(req, res)
- Читает данные (email, password)
- Вызывает authService.register()
- Возвращает ответ (user / error)

login(req, res)
- Читает данные (email, password)
- Вызывает authService.login()
- Устанавливает cookie или возвращает JWT
- Возвращает ответ (user / error)

me(req, res)
- Получает user из req.user (после requireAuth)
- Возвращает данные пользователя

Auth Service
Бизнес-логика аутентификации

register(data)
- Проверяет уникальность email
- Валидирует данные
- Хэширует пароль через passwordService
- Создаёт пользователя (Prisma)
- Возвращает безопасные данные

login(data)
- Находит пользователя по email
- Проверяет пароль (passwordService)
- Создаёт сессию или JWT
- Возвращает user / token

getUserById(id)
- Загружает данные пользователя
- Используется в requireAuth
- Исключает чувствительные поля (например, passwordHash)

passwordService + Prisma Client
Хэширование паролей и работа с базой данных

passwordService

hash(password)
- Генерирует hash (Argon2)
- Возвращает passwordHash

verify(password, hash)
- Проверяет пароль

Prisma Client

prisma.user
- findUnique()
- create()
- update()
- findById()

PostgreSQL
База данных

Таблица users

id	UUID	PK
email	VARCHAR	UNIQUE
passwordHash	VARCHAR	
name	VARCHAR	
role	VARCHAR	
createdAt	TIMESTAMP	
updatedAt	TIMESTAMP	

- В базе хранится только passwordHash, а не пароль
- Используются ограничения UNIQUE, NOT NULL и др.

1 Регистрация (register)
Создание нового пользователя

Запрос с данными → Валидация данных → Хэширование пароля → Создание пользователя → Ответ { user }

```
POST /api/auth/register  { email, password, name }
```

2 Вход (login)
Проверка учётных данных

Поиск пользователя → Проверка пароля → Создание сессии или JWT → Ответ { user / token }

```
POST /api/auth/login  { email, password }
```

3 Защищённые маршруты (requireAuth)
Доступ только для авторизованных пользователей

Проверка токена / сессии → Загрузка пользователя → Доступ разрешён → Protected endpoints

```
router.get('/api/profile', requireAuth, profileController.getProfile)
router.get('/api/orders',  requireAuth, orderController.getOrders)
```

Express Internet Shop — безопасная аутентификация, современный стек, готово к расширению

Регистрация | Вход | Защита маршрутов | Гибкая архитектура

Контрольные вопросы

1. Что проверяет Authentication?
2. Чем Authentication отличается от Authorization?
3. Что такое identifier и credential?
4. Почему HTTP называют stateless?
5. Зачем после login нужен session ID или token?
6. Почему password нельзя сохранять в открытом виде?
7. Почему passwordHash нельзя возвращать frontend?
8. Почему для неизвестного email и неверного password лучше возвращать одинаковую ошибку?
9. Какую роль выполняет requireAuth middleware?
10. Что должен делать frontend после получения 401?

Итоги занятия

1. Authentication подтверждает identity пользователя.
2. Authorization определяет разрешённые действия и изучается отдельно.
3. Обычно пользователь идентифицируется по email и подтверждает identity password.
4. Пароль не хранится в базе — хранится passwordHash.
5. Register создаёт новую учётную запись, login проверяет существующую.
6. HTTP stateless, поэтому после login нужен session или token механизм.
7. Защищённые endpoints используют authentication middleware.
8. Frontend передаёт credentials серверу, но не принимает решение об их корректности.
9. passwordHash никогда не должен попадать в HTTP-ответ.
10. Аутентификация должна использовать HTTPS и безопасную обработку ошибок.

Следующая лекция

- Хеширование паролей
- Почему обычный hash недостаточен для password storage
- Salt
- Argon2id
- Стоимость вычисления
- hash() и verify()
- Практическая реализация passwordService